

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения интернет-магазина спортивных
товаров с системой управления заказами и личными кабинетами
пользователей»

Выполнил: Исаев Арслан Эфлатанович

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Исаеву Арслану Эфлатановичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения интернет-магазина спортивных товаров с системой управления заказами и личными кабинетами пользователей»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих интернет-магазинов спортивных товаров	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и требований к системе	5	02.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	06.04.2026	
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация аутентификации и авторизации пользователей	7	27.04.2026	
Задача 3.3. Реализация системы проверки прав доступа	7	03.05.2026	
Задача 3.4. Реализация CRUD-интерфейса для управления товарами и категориями	12	09.05.2026	
Задача 3.5. Реализация агрегирования данных: статистика продаж, отчёты по заказам	6	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «____» _____ 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«____» _____ 2026 г. _____ А. Э. Исаев
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих интернет-магазинов спортивных товаров	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	11
2.1 Анализ целевой аудитории	11
2.2 Описание функциональности приложения	11
2.3 Проектирование модели данных	11
2.4 Разработка макетов страниц	13
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	14
3.1 Разработка базовой структуры приложения и вёрстка	14
3.2 Реализация аутентификации и авторизации пользователей . .	14
3.3 Реализация системы проверки прав доступа	14
3.4 Реализация CRUD-интерфейса для управления товарами и категориями	15
3.5 Реализация агрегирования данных: статистика продаж, отчёты по заказам	15
ЗАКЛЮЧЕНИЕ	16

ВВЕДЕНИЕ

Актуальность темы. В условиях стремительного роста рынка электронной коммерции спрос на онлайн-покупку спортивных товаров неуклонно увеличивается. Традиционные офлайн-магазины не способны в полной мере удовлетворить потребности современного покупателя, ориентированного на удобный доступ к широкому ассортименту, персонализированные рекомендации и прозрачное управление заказами. Разработка специализированных веб-приложений интернет-магазинов, объединяющих функциональный каталог товаров, систему управления заказами и личные кабинеты пользователей, является необходимым условием для повышения конкурентоспособности компаний в сфере спортивной розничной торговли.

Степень разработанности проблемы. Задача построения интернет-магазинов с полноценной системой управления заказами широко рассматривается в современной литературе по веб-разработке и электронной коммерции. Основы проектирования пользовательских личных кабинетов и систем обработки заказов заложены в работах по архитектуре клиент-серверных приложений и UX-проектированию. В области веб-ориентированных торговых платформ существенный вклад внесли исследователи в сфере баз данных, REST API и систем аутентификации пользователей. Несмотря на наличие готовых коммерческих решений (напр. Shopify, WooCommerce), разработка гибких специализированных приложений для нишевых рынков, таких как спортивные товары, с учётом специфики ассортимента и поведения целевой аудитории, остаётся актуальной и не до конца решённой практической задачей.

Объект исследования — процессы организации и управления онлайн-торговлей спортивными товарами, включая взаимодействие покупателей с интернет-магазином и обработку заказов.

Предмет исследования — программные средства и архитектурные решения для разработки веб-приложения интернет-магазина спортивных товаров, обеспечивающего автоматизацию управления заказами и функциональность личных кабинетов пользователей.

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения интернет-магазина спортивных товаров, обеспечивающего автоматизацию управления заказами, ведение каталога продукции и предоставление пользователям персональных личных кабинетов.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих интернет-магазинов спортивных товаров

На современном рынке интернет-торговли спортивными товарами можно выделить две группы решений: крупные универсальные маркетплейсы и специализированные розничные магазины. Рассмотрим ключевые особенности трёх известных игроков.

Wildberries – международный маркетплейс, предлагающий товары повседневного спроса, включая спортивную одежду, обувь и инвентарь. Платформа объединяет множество продавцов и обеспечивает централизованную обработку заказов. Личный кабинет пользователя позволяет отслеживать покупки, оформлять возвраты, участвовать в акциях. Для малого бизнеса доступна модель размещения товаров с комиссией, однако полноценная автономная система управления заказами и клиентской базой отсутствует – все процессы жёстко интегрированы в инфраструктуру маркетплейса.

Decathlon – вертикально-интегрированный ритейлер, развивающий собственные спортивные бренды. Интернет-магазин предлагает широкий ассортимент товаров для более чем 70 видов спорта, дополненный программой лояльности и персонализированным личным кабинетом. В нём хранится история заказов, адреса доставки, информация о гарантии. Система управления заказами оптимизирована под розничные продажи одной компании и не тиражируется для сторонних продавцов.

SportMaster – российская розничная сеть, чей онлайн-магазин предоставляет доступ к ассортименту спортивных товаров, бонусную программу и личный кабинет с отслеживанием статуса заказа. Платформа обслуживает исключительно собственные операции сети; инструменты для создания независимого магазина или адаптации под нужды небольших специализированных продавцов отсутствуют.

Сравнительная характеристика рассмотренных решений по ключевым критериям приведена в таблице 1.1.

Таблица 1.1 – Сравнительный анализ интернет-магазинов спортивных товаров

Название	Ассортимент	Управление заказами	Личный кабинет	Гибкость для малого бизнеса
Wildberries	Товары всех категорий, включая спорт	Комиссия для продавцов	История заказов, возвраты, избранное	Зависимость от правил маркетплейса, нет автономии
Decathlon	Спорт и активный отдых, собственные бренды	Сквозной цикл «заказ-доставка», самовывоз	Персональные рекомендации, гарантия, лояльность	Не предусмотрена (система только для Decathlon)
SportMaster	Спортивная одежда, обувь, инвентарь	Обработка заказов только для своих продаж	Бонусный счёт, история, статус заказа	Тиражирование для сторонних продавцов отсутствует

Анализ показывает, что все рассмотренные платформы либо являются универсальными маркетплейсами, либо представляют собой закрытые системы крупных розничных сетей. Ни одно из решений не предоставляет малому специализированному магазину спортивных товаров готового инструмента с собственной системой управления заказами, полноценным личным кабинетом пользователя и возможностью гибкой настройки под конкретный ассортимент. Это подтверждает необходимость разработки отдельного веб-приложения, ориентированного на небольшие спортивные магазины и сочетающего функции управления заказами с персональным клиентским сервисом.

1.2 Анализ программных инструментов разработки

Для реализации проекта рассматриваются современные полнофункциональные (Fullstack) подходы. Выбор инструментов обусловлен требованиями к производительности и масштабируемости:

Фронтенд-фреймворк и мета-фреймворк. При выборе инструмента для построения пользовательского интерфейса сравнивались React, Vue.js и

Angular (таблица 1.2).

Таблица 1.2 – Сравнение фронтенд-фреймворков

Критерий	React (выбран)	Vue.js	Angular
Экосистема библиотек для e-commerce	Широкая: React Hook Form, Zustand, React Query, Next.js Commerce	Ограниченная; многие решения не имеют типовой поддержки	Меньше специализированных решений, используется Angular Material с доработками
Производительность и SEO	Next.js с серверным рендерингом (SSR) и статической генерацией (SSG)	Nuxt.js (SSR/SSG), но меньшая популярность для крупных магазинов	Angular Universal для SSR, но настройка сложнее и ресурсоёмкость выше
Кривая обучения и сообщество	Умеренная; огромное сообщество, множество готовых компонентов	Низкая; простота старта, но меньше специалистов	Высокая; требует знания TypeScript, RxJS, DI

React совместно с Next.js обеспечивает серверный рендеринг страниц каталога товаров и маршрутизацию разделов личного кабинета и системы управления заказами без дополнительных надстроек. Широкая экосистема библиотек (React Hook Form для форм оформления заказа, Zustand для управления состоянием корзины) позволяет ускорить разработку интерфейсов электронной коммерции [1].

Бэкенд-платформа. Сравнивались язык Go, Node.js (Express/NestJS) и Python (Django/Flask) по критериям производительности и удобства обработки конкурентных запросов.

- **Go (выбран):** компилируемый язык со встроенной моделью конкурентности (горутины, каналы). Позволяет эффективно обрабатывать тысячи одновременных запросов при минимальном потреблении памяти, что критично для системы управления заказами и личных кабинетов в периоды пиковых нагрузок [2]. Статическая типизация и быстрая сборка снижают количество ошибок на этапе выполнения.
- **Node.js:** событийно-ориентированная модель с асинхронным I/O, но однопоточный цикл обработки событий требует аккуратного управления длительными операциями; масштабирование на многоядерных системах

реализуется через кластеризацию, что усложняет архитектуру.

- **Python:** удобен для прототипирования и работы с данными, однако производительность стандартных веб-фреймворков (Django) уступает Go при высоких нагрузках, а асинхронные возможности (FastAPI) всё ещё требуют тщательной настройки.

Таким образом, Go обеспечивает наилучший баланс между производительностью, простотой конкурентного программирования и удобством развёртывания конечного приложения.

Формат обмена данными. Для взаимодействия клиентской и серверной частей выбран формат JSON (JavaScript Object Notation). В сравнении с XML он обладает меньшей избыточностью, легко читается как человеком, так и машиной, а встроенная поддержка в JavaScript и Go минимизирует накладные расходы на сериализацию/десериализацию. Это особенно важно при передаче структурированных данных о товарах, заказах и профилях пользователей через REST API. Альтернативы, такие как Protocol Buffers (protobuf), обеспечивают более компактное представление, но требуют дополнительных этапов кодогенерации и менее удобны на этапе отладки.

Представленный стек (Next.js/React + Go + JSON) формирует целостную архитектуру, в которой каждый компонент выбран по совокупности критериев: производительности, развитости экосистемы, простоте интеграции и поддержки со стороны сообщества разработчиков.

1.3 Формулировка цели и задач работы

На основе проведенного анализа инструментов (см. подраздел 1.2) и выявленных пробелов в существующих продуктах, была сформулирована цель исследования.

Цель работы — создание веб-приложения интернет-магазина спортивных товаров, обеспечивающего управление каталогом продукции, обработку заказов и предоставление пользователям персональных личных кабинетов.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести сравнительный анализ существующих интернет-магазинов

спортивных товаров и обзор современных технологий разработки веб-приложений;

2. Разработать архитектуру клиент-серверного приложения и спроектировать структуру базы данных для хранения товаров, заказов и данных пользователей;
3. Реализовать программные модули каталога товаров, корзины и системы управления заказами;
4. Реализовать систему личных кабинетов пользователей с возможностью просмотра истории заказов и редактирования профиля;
5. Провести тестирование разработанного веб-приложения и оценить корректность работы основных функциональных модулей.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы являются сотрудники логистических компаний. Выделяются две ключевые роли:

- **Диспетчер-логист** — сотрудник, ответственный за распределение заказов и контроль их выполнения в реальном времени.
- **Курьер** — конечный исполнитель, использующий мобильную версию приложения для навигации по маршруту.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде *User Story*: «Я, как диспетчер, хочу импортировать список адресов, чтобы система автоматически построила кратчайший маршрут для 5 курьеров».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

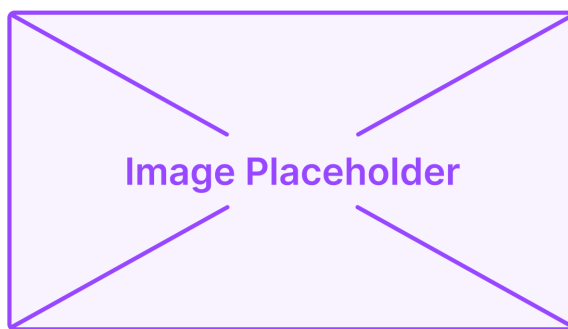


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о произ-

водительности современных СУБД [3].

Структура таблицы заказов представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы заказов (orders)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор заказа
customer_id	UUID	FOREIGN KEY	Ссылка на таблицу клиентов
address	TEXT	NOT NULL	Полный адрес доставки
lat	NUMERIC	NOT NULL	Широта точки (WGS-84)
lng	NUMERIC	NOT NULL	Долгота точки (WGS-84)
weight	FLOAT	CHECK > 0	Вес груза, кг
volume	FLOAT	NOT NULL	Объем груза, м ³
time_from	TIME	NOT NULL	Начало окна доставки
time_to	TIME	NOT NULL	Конец окна доставки
priority	INT	DEFAULT 1	Приоритет (1–5)
status	VARCHAR	'pending'	Статус (ожидание, в пути, выполнен)
courier_id	UUID	NULL	Назначенный исполнитель
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи
updated_at	TIMESTAMP	DEFAULT NOW()	Время последнего изменения
is_fragile	BOOLEAN	FALSE	Хрупкий груз
contact_phone	VARCHAR	NOT NULL	Телефон получателя
price	MONEY	NOT NULL	Стоимость доставки
external_id	VARCHAR	UNIQUE	ID из внешней системы (1C/ERP)

Продолжение таблицы 2.1

Поле	Тип	Ограничение	Описание
notes	TEXT	NULL	Комментарий для курьера
attempts	INT	DEFAULT 0	Количество попыток доставки

Общая стоимость маршрута для службы доставки рассчитывается следующим образом:

$$C_{total} = \sum_{i=1}^n (L_i \cdot P_{fuel} + T_i \cdot W_{hour}) + S_{fixed}, \quad (1)$$

где

L_i — протяженность i -го участка маршрута, км;

P_{fuel} — стоимость топлива на 1 км пути;

T_i — время в пути на участке;

W_{hour} — почасовая ставка оплаты труда курьера;

S_{fixed} — постоянные издержки на обслуживание автопарка.

2.4 Разработка макетов страниц

Макеты страниц

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка

В соответствии с задачей 3.1, была реализована компонентная структура фронтенд-части на базе библиотеки **React.js**. Использование хуков (*Hooks*) позволило эффективно управлять состоянием формы заказа. Пример базового компонента представлен в листинге 1.

Листинг 1 – Пример структуры компонента карточки заказа

```
1 // Комментарий
2 const OrderCard = ({ order }) => {
3   return (
4     <div className="order-card">
5       <h3Заказ> #{order.id}</h3>
6       <pАдрес>: {order.address}</p>
7       <button onClick={() => addToRoute(order.id)}>
8         Добавить в маршрут
9       </button>
10    </div>
11  );
12 };
```

3.2 Реализация аутентификации и авторизации пользователей

Согласно задаче 3.2, в системе реализована ролевая модель доступа. Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При входе в систему сервер генерирует токен, содержащий поле *role*, которое может принимать значения *'dispatcher'* или *'courier'*.

3.3 Реализация системы проверки прав доступа

Ключевым этапом разработки (задача 3.4) стала интеграция API **OpenStreetMap** через библиотеку *Leaflet*. Для решения транспортной задачи был выбран жадный алгоритм с последующей оптимизацией методом локального поиска.

Данные о маршруте передаются в формате JSON (см. листинг 2).

Как видно из листинга 2, каждая точка содержит порядковый номер (*sequence*), что позволяет курьеру следовать по оптимальному пути, рассчитанному на сервере.

Листинг 2 – Структура данных оптимизированного маршрута

```
1 {  
2   "route_id": "r-101",  
3   "courier_id": "u-45",  
4   "waypoints": [  
5     {"lat": 55.75, "lng": 37.61, "sequence": 1},  
6     {"lat": 55.80, "lng": 37.70, "sequence": 2}  
7   ],  
8   "total_distance_km": 12.4  
9 }
```

3.4 Реализация CRUD-интерфейса для управления товарами и категориями

3.5 Реализация агрегирования данных: статистика продаж, отчёты по заказам

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена гипотеза о том, что автоматизация логистических процессов на базе специализированного веб-приложения позволяет существенно повысить эффективность работы служб экспресс-доставки. Теоретический анализ показал, что классическая задача маршрутизации транспортных средств (VRP) в современных условиях требует адаптации под динамические изменения городского трафика и специфику «последней мили».

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование открытых ГИС-технологий и компонентного подхода (React/Node.js) обеспечивает необходимую архитектурную гибкость, позволяя интегрировать алгоритмы оптимизации без зависимости от проприетарных платных сервисов.
2. Проектирование модели данных с поддержкой пространственных расширений (PostGIS) является критическим условием для масштабируемости системы, так как позволяет эффективно обрабатывать геозапросы в режиме реального времени.
3. Внедрение эвристических методов построения маршрутов демонстрирует устойчивый экономический эффект, выраженный в снижении совокупного пробега автотранспорта и сокращении времени планирования рейсов диспетчером.

Рекомендации и предложения. Для практического применения разработанного решения в деятельности малых логистических компаний предлагается использовать модель облачного развертывания. Это обеспечит бесперебойный доступ к интерфейсу управления и мобильному клиенту курьера при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения методов предиктивной аналитики для прогнозирования времени прибытия (ETA) с учетом исторических данных о заторах, а также разработки модуля динамиче-

ской корректировки маршрута при поступлении срочных заявок в процессе выполнения рейса.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [4], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 27.03.2026).
- [2] Донован, Алан А. А. Язык программирования Go [Текст] / Алан А. А. Донован, Брайан У. Керниган. — М. : Вильямс, 2016. — ISBN: 978-5-8459-2051-5.
- [3] Петров, П. П. Анализ производительности современных СУБД [Текст] / П. П. Петров, С. С. Сидоров // Вестник информационных технологий. — 2024. — № 2. — С. 15–22.
- [4] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.